DPMS v2

Doctor-Patient Management System

Comprehensive Technical Documentation

Version 2.0 | March 2026

HMS Hospital | FastAPI + Streamlit + OpenAI Agents SDK

1. Executive Summary

DPMS v2 (Doctor-Patient Management System) is a full-stack hospital management platform designed for small to mid-size hospitals and clinics in India. It handles the complete lifecycle of patient-doctor interactions: from patient registration and appointment booking, through real-time queue management during doctor sessions, to post-visit feedback and ratings.

The system is built on a modern Python stack with FastAPI as the backend API server, PostgreSQL for relational data, MongoDB for AI chat persistence, ChromaDB for vector-based review search (RAG), and Streamlit as the frontend UI. Every role (patient, doctor, nurse, admin) gets a tailored dashboard with role-specific capabilities.

A standout feature is the AI-powered chatbot, built on the OpenAI Agents SDK, which gives each user role a conversational interface with the same capabilities as their dashboard. Patients can book appointments by talking to the chatbot; doctors can manage their queue; nurses can check in patients — all through natural language.

2. System Architecture

2.1 High-Level Architecture

The system follows a clean three-tier architecture with clear separation of concerns:

Layer	Technology	Purpose
Frontend	Streamlit (Python)	Browser-based UI with role-specific dashboards, real-time chat, voice input/output
Backend API	FastAPI (Python)	RESTful API with JWT auth, async PostgreSQL, 9 route modules
AI Layer	OpenAI Agents SDK	One agent per role with function tools matching dashboard capabilities
Primary Database	PostgreSQL	Users, appointments, sessions, queue, audit trails (12 tables)
Chat Storage	MongoDB (Motor)	AI conversation persistence with message compaction
Vector Store	ChromaDB	Doctor review embeddings for semantic search (RAG)
Email	Gmail SMTP (aiosmtplib)	Transactional emails with .ics calendar attachments

2.2 Project Directory Structure

Path	Description
main.py	FastAPI app entry point, lifespan management, router mounting
config.py	Pydantic Settings (env vars: DB, JWT, OpenAI, SMTP, MongoDB, ChromaDB)

database.py	SQLAlchemy async engine, connection pool, auto-migrations
dependencies.py	JWT auth dependency, role-based access control factory
go/models/	Global Objects — long-lived entities (User, Patient, Doctor, Session, etc.)
lo/models/	Life-Cycle Objects — transactional records (Appointment, Waitlist, etc.)
go/services/	Business logic (booking, chat agent, RAG, notifications, email, calendar)
go/adapters/	External integration adapters (Auth/OAuth, Email/SMTP, Calendar/ICS)
api/routes/	FastAPI route handlers (auth, patient, doctor, appointment, queue, admin, chat, rating, session)
api/schemas/	Pydantic request/response models for API validation
streamlit_app.py	Streamlit frontend — all pages, role routing, chat UI (5750 lines)
streamlit_pages/api_client.py	HTTP client wrapper for Streamlit to call FastAPI backend
migrations/	SQL migration files (001-020) for PostgreSQL schema
seed_rag.py	One-time script to embed existing reviews into ChromaDB

2.3 Communication Flow

The frontend (Streamlit) communicates with the backend (FastAPI) exclusively through HTTP REST calls. The flow works as follows:

- User interacts with Streamlit UI (buttons, forms, chat input, voice)
- Streamlit calls api_client.py which makes HTTP requests to FastAPI endpoints on port 8000
- FastAPI routes validate the request, extract the JWT token, and call service layer functions
- Service layer executes business logic, queries PostgreSQL via SQLAlchemy async sessions
- Response travels back: Service returns data to route, route returns JSON to Streamlit, Streamlit renders
- For AI chat: Streamlit sends message to /api/chat/message endpoint, which invokes the OpenAI Agent, which may call function tools that internally call the same FastAPI API endpoints

3. Authentication and Authorization

The system uses JWT (JSON Web Tokens) for stateless authentication with a dual-token architecture:

- Access Token: Short-lived (15 minutes), used for API requests via Bearer header

- Refresh Token: Long-lived (7 days), used to obtain new access tokens without re-login

Authentication flow:

Step	Endpoint	Description
1. Register	POST /api/auth/register	Creates user + patient profile + self-relationship atomically
2. Login	POST /api/auth/login	Email/password verification, returns access + refresh tokens
3. Google OAuth	GET /api/auth/google/login	Redirects to Google, callback creates/links user account
4. Refresh	POST /api/auth/refresh	Exchange refresh token for new access token
5. Me	GET /api/auth/me	Returns current user profile from JWT

Role-Based Access Control (RBAC): Four roles exist — patient, doctor, nurse, admin. The require_role() dependency factory in dependencies.py restricts routes to specific roles. Each route declares which roles can access it. The AI chatbot enforces the same restrictions by giving each role only the tools that match their dashboard.

Password Security: Passwords are hashed using bcrypt via passlib. Google OAuth users have no password (password_hash is null). The system detects Google-only accounts at login and returns a helpful error message.

4. Database Schema (PostgreSQL)

The system uses 12 PostgreSQL tables organized into two categories: Global Objects (GO) for long-lived entities, and Life-Cycle Objects (LO) for transactional records. All tables use UUID primary keys, timestamps, and proper foreign key constraints.

4.1 Global Objects (Long-Lived Entities)

4.1.1 users

Core identity table for all system users (patients, doctors, nurses, admins).

Column	Type	Constraints
id	UUID	PRIMARY KEY, gen_random_uuid()
email	VARCHAR(255)	UNIQUE, NOT NULL
phone	VARCHAR(15)	nullable (family members share numbers)
password_hash	VARCHAR(255)	nullable (null for Google OAuth users)
full_name	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	NOT NULL, CHECK: patient|doctor|nurse|admin
google_id	VARCHAR(255)	UNIQUE, nullable
is_active	BOOLEAN	NOT NULL, DEFAULT true
created_at / updated_at	TIMESTAMPTZ	Auto-managed via trigger

4.1.2 patients

Extended profile for users with the patient role. Links to users via user_id.

Column	Type	Constraints
id	UUID	PRIMARY KEY
user_id	UUID	UNIQUE, FK → users.id
abha_id	VARCHAR(14)	UNIQUE, nullable (India health ID)
date_of_birth	DATE	NOT NULL (used for priority calculation)
gender	VARCHAR(10)	CHECK: male|female|other
blood_group	VARCHAR(5)	nullable
emergency_contact_name/phone	VARCHAR	nullable
address	TEXT	nullable
risk_score	DECIMAL(4,2)	DEFAULT 0.00 (cancellation penalty tracker)

4.1.3 doctors

Doctor profiles linked to users. Stores medical credentials and availability settings.

Column	Type	Purpose
id / user_id	UUID	PK and FK to users
specialization	VARCHAR(255)	Department (e.g., Cardiology, Pediatrics)
qualification	VARCHAR(255)	Medical degree (e.g., MBBS, MD)
license_number	VARCHAR(50)	UNIQUE medical license
consultation_fee	DECIMAL(10,2)	Fee per visit in INR
max_patients_per_slot	INTEGER	DEFAULT 2 (supports overbooking)
is_available	BOOLEAN	Toggle for doctor availability

4.1.4 sessions

Doctor availability windows. A session represents a time block when a doctor sees patients (e.g., 09:00-13:00 morning session). Sessions are the core scheduling unit.

Column	Type	Purpose
id / doctor_id	UUID	PK and FK to doctors
session_date	DATE	Which day this session is for
start_time / end_time	TIME	Session time window
slot_duration_minutes	INTEGER	DEFAULT 15 (each patient slot length)
max_patients_per_slot	INTEGER	DEFAULT 2 (slot_position: 1=regular, 2=overbook)
total_slots	INTEGER	Computed: (end-start)/duration
booked_count	INTEGER	Counter cache for quick availability check
doctor_checkin_at	TIMESTAMPTZ	When doctor actually started (for delay calc)
delay_minutes	INTEGER	Auto-computed from checkin vs start_time
status	VARCHAR	active|cancelled|completed|inactive

4.1.5 Other GO Tables

patient_relationships: Enables multi-beneficiary booking. A patient (booker) can book appointments for family members (beneficiaries). Self-relationships are auto-approved; others require approval. Columns: booker_patient_id, beneficiary_patient_id, relationship_type (self, spouse, child, parent, sibling, friend, other), is_approved.

scheduling_config: Key-value JSONB store for system-wide settings. Configurable by admins. Defaults: max_bookings_per_day=5, max_bookings_per_week=15, cancel_cooldown_hours=2, risk_score_threshold=7.0, risk_decay_per_day=0.1.

4.2 Life-Cycle Objects (Transactional Records)

4.2.1 appointments

The core transactional table. Each row is one patient-session assignment with full lifecycle tracking.

Column	Type	Purpose
session_id	UUID	Which doctor session this appointment belongs to
patient_id	UUID	The beneficiary patient being seen
booked_by_patient_id	UUID	Who made the booking (may differ from patient_id)
slot_number	INTEGER	Which time slot (1-based, 0 for emergency)
slot_position	INTEGER	1=regular, 2=overbook, 3=emergency
priority_tier	VARCHAR	NORMAL|HIGH|CRITICAL (age-based, immutable)
visual_priority	INTEGER	1-10, adjustable by staff for queue ordering
is_emergency	BOOLEAN	Emergency flag for special handling
status	VARCHAR	booked → checked_in → in_progress → completed (or cancelled|no_show)
checked_in_at / checked_in_by	TIMESTAMPTZ/UUID	When and by whom the patient was checked in

4.2.2 Other LO Tables

waitlist: When all slots are full, patients join the waitlist. On cancellation, the highest-priority waitlist entry is auto-promoted to a real appointment. Status: waiting → promoted (or cancelled|expired).

cancellation_log: Immutable audit trail of every cancellation. Records who cancelled, when, the reason, the risk_delta penalty, and hours_before_appointment. Used for risk score calculations.

booking_audit_log: Complete audit trail of all booking-related actions. Tracks: book, cancel, check_in, complete, no_show, waitlist_add, waitlist_promote, emergency_book, staff_book, delay_update, session_cancel. Stores JSONB metadata and optional IP address.

notification_log: Tracks all email/SMS/push notifications. Status: pending → sent (or failed). Used to verify delivery and debug notification issues.

doctor_ratings: Patient feedback after completed appointments. Stores 1-5 star rating, optional text review, and AI-computed sentiment_score (-1.0 to 1.0). One rating per appointment (UNIQUE constraint on appointment_id). Triggers RAG embedding on submission.

5. Booking Engine (Business Logic)

The booking_service.py is the brain of the appointment system. It implements a sophisticated multi-step booking flow with safety checks, rate limiting, and automatic waitlist management.

5.1 Booking Flow (8 Steps)

When a patient (or staff on behalf of a patient) books an appointment, the system executes these steps in order:

Step	Check	What Happens on Failure
1	Risk score check (≥ 7.0 blocks booking)	HTTP 403: Account restricted due to cancellation history
2	Daily rate limit (max 5/day per booker)	HTTP 429: Daily booking limit reached
3	Weekly rate limit (max 15/week per booker)	HTTP 429: Weekly booking limit reached
4	Relationship validation (booker must be approved for beneficiary)	HTTP 403: Not authorized to book for this patient
5	Session locking (SELECT FOR UPDATE to prevent race conditions)	Concurrent requests wait in queue
6	Slot availability (find next open slot with position < max_per_slot)	Falls through to waitlist
7	Create appointment + update booked_count + audit log	Atomic transaction (all or nothing)
8	Fire email notification (non-blocking, never fails the booking)	Logged but booking succeeds

5.2 Priority System

The system uses a dual-priority model for queue ordering:

Age-Based Priority (immutable): Calculated at booking time from date_of_birth. Children under 12 and seniors over 60 get CRITICAL priority. Teens (12-18) and older adults (50-60) get HIGH. Everyone else gets NORMAL. This never changes after booking.

Visual Priority (staff-adjustable): A 1-10 scale that nurses and doctors can modify in real-time to handle clinical urgency. Default is 5. A patient who looks unwell can be bumped to 9-10 without changing their age-based tier.

Queue Ordering: is_emergency DESC, priority_tier DESC (CRITICAL > HIGH > NORMAL), visual_priority DESC, created_at ASC. Emergency patients always come first, then by clinical priority, then by arrival time.

5.3 Risk Score and Cancellation Penalties

To discourage last-minute cancellations, the system maintains a risk_score per patient. Penalties increase based on how close to the appointment the cancellation happens:

Time Before Appointment	Risk Penalty	Severity
Less than 2 hours	+3.0	Severe (almost a no-show)
2-6 hours	+2.0	Moderate
6-24 hours	+1.0	Mild
More than 24 hours	+0.5	Minimal

When risk_score reaches 7.0 or higher, the patient is blocked from making new bookings. Risk scores decay by 0.1 per day (configurable), implemented via a nightly cron job that calls PatientModel.decay_all_risk_scores().

5.4 Waitlist and Auto-Promotion

When all slots in a session are full, the patient is added to the waitlist instead of being rejected. Waitlist entries are prioritized using the same priority_tier system (CRITICAL > HIGH > NORMAL, then by creation time).

When any appointment is cancelled, the system automatically promotes the highest-priority waitlist entry: creates a new appointment in the freed slot, sends a booking confirmation email, and marks the waitlist entry as promoted. This happens atomically within the cancellation transaction.

6. Real-Time Queue Management

The queue system manages the live patient flow during a doctor session. It handles the journey from check-in to consultation completion.

6.1 Appointment Status Lifecycle

Status	Meaning	Triggered By
booked	Appointment confirmed, patient not yet at clinic	Booking or waitlist promotion
checked_in	Patient has arrived at the clinic	Nurse/admin check-in action
in_progress	Patient is currently being seen by the doctor	Doctor calls patient (call-next or call-patient)
completed	Consultation finished	Doctor marks complete
cancelled	Appointment cancelled before session	Patient or staff cancellation
no_show	Patient did not arrive; session ended	Automatic when doctor ends session

6.2 Queue Operations

These operations are available to doctors and nurses during an active session:

- Check-in: Nurse checks patient in at arrival. Records checked_in_at timestamp and checked_in_by user.
- Call Next: Doctor calls the next patient from the queue (ordered by priority). Changes status from checked_in to in_progress.
- Call Specific Patient: Doctor calls a specific patient out of order (for emergencies or clinical judgment).
- Complete: Doctor marks current consultation as done. Records completed_at and optional duration_minutes.
- Escalate: Increase a patient's visual_priority to move them up in the queue.
- End Session: Doctor ends the session. All remaining booked patients become no_show, checked_in patients become cancelled. Blocks if any patient is in_progress.

7. Notification System

The system sends transactional emails at key points in the appointment lifecycle. All notifications are fire-and-forget: they never block or fail the main operation.

7.1 Email Types

Trigger	Email Type	Includes
Appointment booked	Booking Confirmation	Doctor name, date, slot time, .ics calendar attachment
Appointment cancelled	Cancellation Notice	Reason, risk penalty (if any), .ics CANCEL event
Doctor running late	Delay Notification	Updated delay minutes, sent to all waiting patients
Session cancelled	Session Cancellation	Sent to all patients in that session
Session ended (no-shows)	No-Show Notice	Informs patient of missed appointment + risk penalty
Check-in reminder	Reminder	Sent before appointment time

7.2 Architecture Pattern

The notification system uses an adapter pattern for extensibility:

- EmailAdapter (abstract interface): Defines send(), is_configured(), send_background() methods
- GmailEmailAdapter: Concrete implementation using aiosmtplib with TLS on port 587
- CalendarAdapter + ICSCalendarAdapter: Generates RFC 5545 .ics calendar events attached to emails

- NotificationDispatcher: Orchestrates email sending after operations. Fetches all needed context while the DB session is still alive, then sends asynchronously.

Emails to @dpms.local test addresses are silently skipped. The pattern makes it easy to swap Gmail for SendGrid, SES, or add SMS via Twilio.

8. AI Chatbot (OpenAI Agents SDK)

The AI chatbot is one of the system's most powerful features. Every user role gets a conversational assistant that can perform the same actions as their dashboard. The chatbot is built on the OpenAI Agents SDK with a strict one-agent-per-role architecture (no sub-agents, no handoffs).

8.1 Architecture Principles

- One agent per role: Patient, Doctor, Nurse, Admin each get their own agent with role-specific tools
- Tools mirror dashboard: Every button/action on the dashboard has a corresponding function_tool for the agent
- Same API backend: Agent tools call the same FastAPI endpoints as the Streamlit UI (via internal HTTP)
- Memory via compaction: OpenAIResponsesCompactionSession automatically summarizes old messages to save tokens. Recent messages are kept verbatim, older ones are compressed into summaries.
- One conversation per user: All messages stored under the user's UUID, regardless of topic. New Chat clears everything.

8.2 Agent Tools by Role

Tool Category	Tools	Available To
Information	list_departments, list_doctors, get_doctor_details, list_sessions	All roles
Patient Booking	book_appointment, cancel_appointment, my_appointments, my_profile, add_family_member	Patient
Queue Management	checkin_patient, call_next_patient, call_specific_patient, complete_patient, escalate_patient	Doctor, Nurse
Session Control	start_session, end_session, update_delay, list_sessions_for_doctor	Doctor, Nurse
Staff Booking	book_for_patient, emergency_book, list_patients_search	Nurse, Admin
Rating/Feedback	submit_rating, get_doctor_ratings, get_doctor_rating_stats, search_feedback	All roles
Admin	All tools + user management, config, audit logs	Admin

8.3 System Prompt Engineering

Each agent receives a carefully crafted system prompt that includes:

- Role identity and behavioral guidelines (professional, no hallucination, use tools not guesswork)
- Mandatory reminders injected into every user message: current date, user context (name, ID, patient ID), and instructions to ALWAYS use tools for real-time data
- Form context: When the user is on a specific page (e.g., appointment booking), the current form state is injected so the agent can reference it
- Rate limiting: Maximum 20 turns per session to prevent runaway conversations

8.4 Chat Storage (MongoDB)

Conversation history is persisted in MongoDB using two collections:

Collection	Schema	Purpose
chat_sessions	{session_id, created_at, updated_at}	Session metadata, unique index on session_id
chat_messages	{session_id, seq, message_data, created_at}	Individual messages with auto-incrementing sequence numbers

MongoSession implements the OpenAI Agents SDK SessionABC interface (get_items, add_items, pop_item, clear_session) as a drop-in replacement for SQLiteSession. The session_id is the user's UUID, so all conversations for a user are stored together regardless of topic.

The compaction layer (OpenAIResponsesCompactionSession) wraps MongoSession and automatically summarizes older messages. This keeps the context window manageable while preserving conversation continuity across server restarts and logins.

8.5 Voice Features (STT/TTS)

The chatbot supports voice input and output:

- Speech-to-Text (STT): Uses audio_recorder_streamlit widget to capture audio in the browser, sends to /api/chat/transcribe endpoint which calls OpenAI Whisper API
- Text-to-Speech (TTS): After each assistant response, converts text to audio via OpenAI TTS API (tts-1 model, alloy voice). Audio bytes are stored in st.session_state._pending_tts_audio and played via st.audio(autoplay=True) on the next Streamlit rerun cycle

9. RAG Pipeline (Feedback/Rating System)

RAG (Retrieval-Augmented Generation) enables semantic search over patient reviews. Instead of keyword matching, the system understands the meaning of queries like "complaints about long wait times" and finds relevant reviews even if they don't contain those exact words.

9.1 How It Works

The RAG pipeline has three stages:

Stage 1 — Embed on Submit: When a patient submits a review, the system builds a rich document (e.g., “Doctor: Dr. Ananya. Rating: 4/5. Review: Very thorough examination...”), generates an embedding vector using OpenAI’s text-embedding-3-small model, and stores it in ChromaDB along with metadata (doctor_id, rating, sentiment_score, etc.).

Stage 2 — Sentiment Analysis: Before embedding, GPT-4o-mini scores the review text from -1.0 (very negative) to 1.0 (very positive). This sentiment_score is stored both in PostgreSQL (doctor_ratings table) and as ChromaDB metadata for filtering.

Stage 3 — Search on Query: When someone searches for feedback (via the AI chatbot’s search_feedback tool or the API), the query is embedded using the same model, and ChromaDB performs cosine similarity search to find the most semantically relevant reviews. Results can be filtered by doctor_id and minimum rating.

9.2 ChromaDB Details

Property	Value
Collection name	doctor_reviews
Similarity metric	cosine
Embedding model	text-embedding-3-small (OpenAI)
Storage	Persistent on disk (./chroma_data/)
Document stored	Raw review text
Metadata stored	doctor_id, patient_id, appointment_id, rating, sentiment_score, doctor_name, created_at
ID	Same UUID as doctor_ratings.id in PostgreSQL (ensures 1:1 mapping)

9.3 Rating Submission Flow

The complete flow when a patient rates a completed appointment:

- Patient clicks “Rate this visit” on a completed appointment in the Streamlit UI
- Selects 1-5 star rating and optionally writes a text review
- Frontend calls POST /api/ratings with appointment_id, rating, review
- Backend validates: appointment exists, is completed, not already rated, patient owns it
- If review text exists: GPT-4o-mini computes sentiment score

- Rating saved to PostgreSQL (doctor_ratings table)
- If review text exists: Review embedded in ChromaDB (non-blocking, failure doesn’t fail the request)
- Response returned to frontend with rating details

10. Frontend (Streamlit)

The entire frontend is a single Streamlit application (streamlit_app.py, ~5750 lines) that dynamically renders different dashboards based on the authenticated user's role. It communicates with the FastAPI backend exclusively through the api_client.py HTTP wrapper.

10.1 Pages by Role

Role	Pages	Key Capabilities
Patient	Dashboard, Book Appointment, My Appointments, Profile, Chat	View doctors, book/cancel appointments, rate visits, manage family, AI chat
Doctor	Dashboard (date picker + queue), My Queue, Session Controls, Chat	View/manage queue, call patients, complete consultations, end sessions, AI chat
Nurse	Session & Queue, Emergency Book, Chat	Check-in patients, manage queue, book on behalf of patients, emergency bookings, AI chat
Admin	All nurse pages + User Management, Config, Audit Logs, Chat	Full system control: manage users, doctors, scheduling config, view audit trails, AI chat

10.2 Key Frontend Patterns

Session State Management: Streamlit reruns the entire script on every interaction. State is preserved in st.session_state with keys for access_token, refresh_token, user profile, page navigation, and transient UI state (open forms, pending audio, etc.).

Auto-refresh on 401: When the access token expires (15-minute window), api_client.py automatically attempts a refresh using the stored refresh_token. If refresh also fails, the user is logged out.

Direct DB Queries: For session management (doctor/nurse views), the frontend makes direct PostgreSQL queries via psycopg2 to fetch all sessions regardless of status. This bypasses the API which only returns active sessions. This is a known temporary pattern until the API supports include_all filtering.

Guard Conditions: Two guard conditions control button visibility: can_act_today (is_today AND status=active) for queue buttons, and can_add_patient (is_today_or_future AND status=active) for Add Patient. This prevents actions on past or inactive sessions.

11. API Endpoints Reference

All endpoints are prefixed with /api/ and require JWT Bearer authentication (except auth endpoints). The full interactive documentation is available at /docs (Swagger UI) when the server is running.

Module	Prefix	Key Endpoints
Auth	/api/auth	register, login, google/login, google/callback, refresh, me
Patients	/api/patients	profile (GET/PUT), relationships, family-members, search
Doctors	/api/doctors	list (GET), {id} details, {id}/sessions
Appointments	/api/appointments	book, cancel, departments, staff-book, emergency-book, reassign
Sessions	/api/sessions	checkin, delay, extend, complete, activate, deactivate, cancel
Queue	/api/queue	checkin, call-patient, call-next, escalate, complete, undo variants
Admin	/api/admin	users, doctors (CRUD), config (GET/PUT), audit-logs, patients
Chat	/api/chat	message, transcribe, tts, clear, history, health
Ratings	/api/ratings	submit (POST), doctor/{id} (GET), doctor/{id}/stats (GET)

12. Deployment and Setup

12.1 Prerequisites

- Python 3.10+ with pip
- Node.js (not required for runtime, only for document generation)
- PostgreSQL 14+ running on localhost:5432
- MongoDB 6+ running on localhost:27017
- OpenAI API key (for AI chatbot, embeddings, sentiment analysis, STT/TTS)
- Gmail App Password (for email notifications, optional)

12.2 Setup Steps

1. Clone the repository and create a virtual environment:

```
python3 -m venv venv && source venv/bin/activate
```

2. Install Python dependencies:

```
pip install -r requirements.txt
```

3. Copy .env.example to .env and fill in your credentials (DATABASE_URL, OPENAI_API_KEY, MONGODB_URL, SMTP credentials).

4. Run database migrations:

```
bash run_migrations.sh
```

5. Seed the RAG vector store (after migration 020):

```
python3 seed_rag.py
```

6. Start the FastAPI backend:

```
uvicorn main:app --reload --port 8000
```

7. Start the Streamlit frontend (in a separate terminal):

```
streamlit run streamlit_app.py
```

12.3 Key Configuration (config.py / .env)

Variable	Default	Des
DATABASE_URL	postgresql+asyncpg://postgres:postgres@localhost:5432/dpms_v2	Post conn strin
MONGODB_URL	mongodb://localhost:27017	Mon conn strin
MONGODB_DATABASE	dpms_v2_chat	Mon data nam chat
OPENAI_API_KEY	(required)	Ope key f ager emb TTS
OPENAI_EMBEDDING_MODEL	text-embedding-3-small	Mod for r emb
CHROMA_PERSIST_DIRECTORY	./chroma_data	On-c stora for Chro
JWT_SECRET_KEY	change-me-in-production	Secr signi toke

JWT_ACCESS_TOKEN_EXPIRE_MINUTES	15	Acce toke lifeti
JWT_REFRESH_TOKEN_EXPIRE_DAYS	7	Refr toke lifeti
SMTP_HOST / SMTP_PORT	smtp.gmail.com / 587	Ema conf
APP_PORT	8000	Fast serv

13. Technology Stack Summary

Category	Technology	Version	Why This Choice
Backend Framework	FastAPI	0.115.0	Async Python, auto-generated docs, dependency injection
Frontend	Streamlit	1.40.0	Rapid Python UI, built-in session state, widgets
Primary DB	PostgreSQL	14+	ACID transactions, JSONB, partial indexes, row-level locking
Async DB Driver	asyncpg + SQLAlchemy	0.30.0 / 2.0.35	Non-blocking DB access, connection pooling
Chat Storage	MongoDB (Motor)	3.6.0	Document-shaped data, flexible schema, async driver
Vector Store	ChromaDB	0.5.0	Simple embedded vector DB, cosine similarity, metadata filtering
AI Framework	OpenAI Agents SDK	0.0.7	Function calling, session management, compaction
AI Models	GPT-4o / GPT-4o-mini	Latest	Agent reasoning, sentiment analysis
Embeddings	text-embedding-3-small	Latest	Review vectorization for RAG
Voice (STT)	OpenAI Whisper	Latest	Speech-to-text for chat input
Voice (TTS)	OpenAI TTS (tts-1)	Latest	Text-to-speech for chat responses
Auth	JWT (python-jose)	3.3.0	Stateless auth with refresh tokens
OAuth	Google Auth	2.35.0	Google sign-in support
Email	aiosmtplib	3.0.0	Async SMTP with TLS
Calendar	RFC 5545 / ICS	Custom	.ics attachments for email notifications

— End of Documentation —